

# EDEMY LMS

## Full Stack Project

Complete Setup & Run Guide

|                 |                                        |
|-----------------|----------------------------------------|
| <b>Project</b>  | Edemy LMS (Learning Management System) |
| <b>Stack</b>    | React + Node.js + Express + MongoDB    |
| <b>Auth</b>     | Clerk                                  |
| <b>Payments</b> | Stripe                                 |
| <b>Media</b>    | Cloudinary                             |
| <b>Tunnel</b>   | Ngrok                                  |
| <b>Author</b>   | Samruddha Shewale                      |

This guide provides step-by-step instructions to run the Edemy LMS project from scratch on your local machine.

# Table of Contents

- 1. Prerequisites
- 2. Project Folder Structure
- 3. Step 1 — Configure Server Environment (.env)
- 4. Step 2 — Set Up MongoDB (Local)
- 5. Step 3 — Set Up Cloudinary
- 6. Step 4 — Set Up Clerk Authentication
- 7. Step 5 — Set Up Stripe Payments
- 8. Step 6 — Configure Client Environment (.env)
- 9. Step 7 — Install Dependencies
- 10. Step 8 — Run Ngrok
- 11. Step 9 — Configure Clerk Webhook
- 12. Step 10 — Configure Stripe Webhook
- 13. Step 11 — Start the Project
- 14. Daily Startup Checklist
- 15. Troubleshooting Common Errors
- 16. Manual Enrollment Script

# 1. Prerequisites

Before running the project, make sure you have the following installed on your Windows computer:

| Software        | Download Link                                                                                               | Why Needed                      |
|-----------------|-------------------------------------------------------------------------------------------------------------|---------------------------------|
| Node.js v18+    | <a href="https://nodejs.org">https://nodejs.org</a>                                                         | Run backend server and frontend |
| VS Code         | <a href="https://code.visualstudio.com">https://code.visualstudio.com</a>                                   | Code editor                     |
| MongoDB (local) | <a href="https://www.mongodb.com/try/download/community">https://www.mongodb.com/try/download/community</a> | Local database                  |
| MongoDB Compass | <a href="https://www.mongodb.com/try/download/compass">https://www.mongodb.com/try/download/compass</a>     | Database GUI                    |
| Ngrok           | <a href="https://ngrok.com/download">https://ngrok.com/download</a>                                         | Expose local server to internet |
| Git (optional)  | <a href="https://git-scm.com">https://git-scm.com</a>                                                       | Version control                 |

You also need free accounts on these platforms:

| Platform                 | URL                                                                                 | Purpose                                     |
|--------------------------|-------------------------------------------------------------------------------------|---------------------------------------------|
| MongoDB Atlas (optional) | <a href="https://cloud.mongodb.com">https://cloud.mongodb.com</a>                   | Cloud database (project uses local MongoDB) |
| Cloudinary               | <a href="https://cloudinary.com">https://cloudinary.com</a>                         | Image and video uploads                     |
| Clerk                    | <a href="https://clerk.com">https://clerk.com</a>                                   | User authentication                         |
| Stripe                   | <a href="https://stripe.com">https://stripe.com</a>                                 | Payment processing                          |
| Ngrok                    | <a href="https://dashboard.ngrok.com/signup">https://dashboard.ngrok.com/signup</a> | Webhook tunneling                           |

## 2. Project Folder Structure

Your project is located at:

```
D:\OneDrive\Desktop\lms\LMS-full-stack\lms-full-stack\
```

The structure looks like this:

```
lms-full-stack/  
  client/                (React Frontend)  
    src/  
      pages/student/    (Home, CoursesList, Player, About, Contact, Privacy)  
      pages/educator/   (Dashboard, AddCourse, MyCourses)  
      components/       (Navbar, Footer, CourseCard, etc.)  
      context/          (AppContext.jsx)  
      .env              (Frontend environment variables)  
  server/                (Node.js Backend)  
    controllers/        (courseController, educatorController, etc.)  
    models/             (User, Course, Purchase, CourseProgress)  
    routes/             (courseRoute, educatorRoutes, userRoutes)  
    middlewares/        (authMiddleware.js)  
    configs/            (mongodb.js, cloudinary.js, multer.js)  
    .env                (Backend environment variables)  
    server.js           (Main server entry point)
```

### 3. Step 1 — Configure Server Environment (.env)

Open this file in VS Code:

```
D:\OneDrive\Desktop\lms\LMS-full-stack\lms-full-stack\server\.env
```

Make sure it contains ALL of the following values (filled in — not placeholders):

```
CURRENCY=USD

# MongoDB Setup
MONGODB_URI=mongodb://127.0.0.1:27017/lms

# Cloudinary Setup
CLOUDINARY_NAME=your_cloud_name
CLOUDINARY_API_KEY=your_api_key
CLOUDINARY_SECRET_KEY=your_api_secret

# Clerk Setup
CLERK_WEBHOOK_SECRET=whsec_XXXXXXXXXX
CLERK_PUBLISHABLE_KEY=pk_test_XXXXXXXXXX
CLERK_SECRET_KEY=sk_test_XXXXXXXXXX

# Stripe Setup
STRIPE_WEBHOOK_SECRET=whsec_XXXXXXXXXX
STRIPE_PUBLISHABLE_KEY=pk_test_XXXXXXXXXX
STRIPE_SECRET_KEY=sk_test_XXXXXXXXXX
```

**Note:** Never use quotes or spaces around the = sign. Wrong: KEY = 'value'. Correct: KEY=value

## 4. Step 2 — Set Up MongoDB (Local)

This project uses a LOCAL MongoDB database (not Atlas). Make sure MongoDB is installed and running on your computer.

### Check if MongoDB is running:

Open Task Manager (Ctrl+Shift+Esc) and look for 'MongoDB' in the Services tab. If not running, start it:

```
# Press Windows Key, search for 'Services'  
# Find 'MongoDB Server' and click 'Start'
```

### Verify the connection:

Open MongoDB Compass and connect using:

```
mongodb://127.0.0.1:27017
```

You should see the 'lms' database after running the project for the first time.

**Success:** MongoDB is working if you see 'Database Connected Successfully!' in the server terminal.

## 5. Step 3 — Set Up Cloudinary

Cloudinary is used to store course thumbnail images. Follow these steps:

- Go to <https://cloudinary.com> and log in
- Click on your profile icon in the top right
- Go to 'Settings' → 'API Keys' tab
- You will see: Cloud Name, API Key, and API Secret
- Click the eye icon next to API Secret to reveal it
- Copy all three values into your server/.env file:

```
CLOUDINARY_NAME=dmkwf9hm5  
CLOUDINARY_API_KEY=527535437646149  
CLOUDINARY_SECRET_KEY=WJ8y4R2_x0TptAge8mayqWSBDhU
```

**Note:** The CLOUDINARY\_SECRET\_KEY must match the variable name exactly — not CLOUDINARY\_API\_SECRET.

## 6. Step 4 — Set Up Clerk Authentication

Clerk handles user signup, login, and session management.

### Get Clerk API Keys:

- Go to <https://clerk.com> and log in
- Select your 'LMS Project' application
- Click 'API Keys' in the left sidebar
- Copy 'Publishable Key' (starts with pk\_test\_...)
- Copy 'Secret Key' (starts with sk\_test\_...)
- Add both to server/.env and the publishable key to client/.env

```
# In server/.env
CLERK_PUBLISHABLE_KEY=pk_test_xxxxxxxxxx
CLERK_SECRET_KEY=sk_test_xxxxxxxxxx

# In client/.env
VITE_CLERK_PUBLISHABLE_KEY=pk_test_xxxxxxxxxx
```

### Clerk Webhook Secret:

The webhook secret is obtained when you set up the webhook (see Step 9). You will update CLERK\_WEBHOOK\_SECRET at that point.



## 7. Step 5 — Set Up Stripe Payments

Stripe handles course payments. The project uses Stripe in TEST mode.

### Get Stripe API Keys:

- Go to <https://dashboard.stripe.com> and log in
- Make sure you are in 'Test mode' (toggle in top right)
- Click 'Developers' in the top right
- Click 'API keys' in the left sidebar
- Copy 'Publishable key' (starts with pk\_test\_...)
- Copy 'Secret key' (starts with sk\_test\_...)
- Add both to server/.env

```
# In server/.env
STRIPE_PUBLISHABLE_KEY=pk_test_XXXXXXXXXX
STRIPE_SECRET_KEY=sk_test_XXXXXXXXXX
```

**Note:** STRIPE\_WEBHOOK\_SECRET is added after setting up the webhook in Step 10.

### Test Card for Payments:

| Field       | Value                   |
|-------------|-------------------------|
| Card Number | 4242 4242 4242 4242     |
| Expiry Date | 12/34 (any future date) |
| CVC         | 123 (any 3 digits)      |
| Name        | Any name                |
| Country     | Any                     |

## 8. Step 6 — Configure Client Environment (.env)

Open this file in VS Code:

```
D:\OneDrive\Desktop\lms\LMS-full-stack\lms-full-stack\client\.env
```

Make sure it contains exactly this (no quotes, no spaces around =):

```
VITE_CLERK_PUBLISHABLE_KEY=pk_test_XXXXXXXXXX  
VITE_BACKEND_URL=http://localhost:5000  
VITE_CURRENCY=$
```

**Note:** Replace `pk_test_XXXXXXXXXX` with your actual Clerk Publishable Key. `VITE_BACKEND_URL` must point to port 5000 where your server runs.

## 9. Step 7 — Install Dependencies

You need to install npm packages for both the server and client. Do this ONCE when setting up the project for the first time, or if node\_modules is missing.

### Install Server Dependencies:

```
cd D:\OneDrive\Desktop\lms\LMS-full-stack\lms-full-stack\server  
npm install
```

### Install Client Dependencies:

```
cd D:\OneDrive\Desktop\lms\LMS-full-stack\lms-full-stack\client  
npm install
```

**Success:** Both installs should complete without errors. This takes 2-5 minutes each.

## 10. Step 8 — Run Ngrok

Ngrok creates a public URL that forwards to your local server. This is required for Clerk and Stripe webhooks to work.

### First time setup (do this once):

- Go to <https://dashboard.ngrok.com/signup> and create a free account
- After login, go to <https://dashboard.ngrok.com/get-started/your-authtoken>
- Copy your authtoken
- Open a terminal and run:

```
ngrok config add-authtoken YOUR_TOKEN_HERE
```

### Every time you run the project:

```
ngrok http 5000
```

You will see output like:

|                |                                                                                                                                    |
|----------------|------------------------------------------------------------------------------------------------------------------------------------|
| Session Status | online                                                                                                                             |
| Forwarding     | <a href="https://xxxx.ngrok-free.app">https://xxxx.ngrok-free.app</a> -> <a href="http://localhost:5000">http://localhost:5000</a> |

**Note:** The ngrok URL CHANGES every time you restart ngrok. You must update it in both Clerk and Stripe webhook settings each time.

## 11. Step 9 — Configure Clerk Webhook

The Clerk webhook saves new users to your MongoDB database when they sign up.

### Setup Steps:

- Go to <https://clerk.com> and log in to your LMS Project
- Click 'Configure' tab at the top
- Click 'Webhooks' in the left sidebar
- Click your existing endpoint or 'Add Endpoint'
- Set the URL to your ngrok URL + /clerk:

```
https://xxxx.ngrok-free.app/clerk
```

Make sure these events are selected:

- user.created
- user.updated
- user.deleted

After saving, copy the Signing Secret and update server/.env:

```
CLERK_WEBHOOK_SECRET=whsec_xxxxxxxxxx
```

**Note:** Every time ngrok restarts with a new URL, update this webhook endpoint URL in the Clerk dashboard.

## 12. Step 10 — Configure Stripe Webhook

The Stripe webhook automatically enrolls users in courses after successful payment.

### Setup Steps:

- Go to <https://dashboard.stripe.com>
- Click 'Developers' in the top right
- Click 'Webhooks' in the left sidebar
- Click your existing endpoint or 'Add destination'
- Set the URL to your ngrok URL + /stripe:

```
https://xxxx.ngrok-free.app/stripe
```

Make sure this event is selected:

- checkout.session.completed

After saving, copy the Signing Secret and update server/.env:

```
STRIPE_WEBHOOK_SECRET=whsec_XXXXXXXXXX
```

**Note:** Every time ngrok restarts with a new URL, update this webhook endpoint URL in the Stripe dashboard too.

## 13. Step 11 — Start the Project

Open VS Code and open 3 separate terminals (press Ctrl+Shift+` three times). Run each command in a different terminal:

### Terminal 1 — Backend Server:

```
cd D:\OneDrive\Desktop\lms\LMS-full-stack\lms-full-stack\server
node server.js
```

**Success:** Database Connected Successfully! Server is running on port 5000

### Terminal 2 — Frontend Client:

```
cd D:\OneDrive\Desktop\lms\LMS-full-stack\lms-full-stack\client
npm run dev
```

**Success:** VITE ready Local: http://localhost:5173

### Terminal 3 — Ngrok Tunnel:

```
ngrok http 5000
```

**Success:** Session Status: online Forwarding: https://xxxx.ngrok-free.app

### Open in Browser:

```
http://localhost:5173
```

**Note:** All 3 terminals must stay open while you use the project. Closing any one will break the app.

## 14. Daily Startup Checklist

Every time you want to work on or demo the project, follow this checklist:

| # | Task                      | Command / Action                                |
|---|---------------------------|-------------------------------------------------|
| 1 | Start MongoDB service     | Services → MongoDB Server → Start               |
| 2 | Start backend server      | cd server → node server.js                      |
| 3 | Start frontend client     | cd client → npm run dev                         |
| 4 | Start ngrok               | ngrok http 5000                                 |
| 5 | Update Clerk webhook URL  | Clerk Dashboard → Webhooks → Edit endpoint URL  |
| 6 | Update Stripe webhook URL | Stripe Dashboard → Webhooks → Edit endpoint URL |
| 7 | Open browser              | http://localhost:5173                           |



## 15. Troubleshooting Common Errors

### **Error: Error: listen EADDRINUSE :::5000**

**Cause:** Port 5000 is already in use by another process.

```
netstat -ano | findstr :5000
taskkill /PID /F
Then run: node server.js
```

### **Error: Network Error (on website)**

**Cause:** The backend server is not running.

```
Go to the node terminal and run: node server.js
Wait for 'Database Connected Successfully!'
```

### **Error: User Not Found (when clicking Become Educator)**

**Cause:** Your user was not saved to MongoDB (webhook issue).

```
Run the manual user creation script from the conversation
or sign out and sign in again after setting up the Clerk webhook.
```

### **Error: Must supply api\_secret (when adding course)**

**Cause:** The Cloudinary secret key is wrong or missing.

```
Check server/.env – make sure CLOUDINARY_SECRET_KEY is correct.
Note: The variable is CLOUDINARY_SECRET_KEY not CLOUDINARY_API_SECRET.
```

### **Error: Webhook signature error: Timestamp outside tolerance**

**Cause:** You are resending old Stripe events. This is expected.

```
Do NOT resend old events. Instead make a fresh payment
with test card 4242 4242 4242 4242.
```

### **Error: Courses not showing on homepage**

**Cause:** No courses in database or server not running.

1. Make sure server is running (node server.js)
2. Go to Educator Dashboard → Add Course
3. Add at least one course with thumbnail

### **Error: Cannot find module 'server.js'**

**Cause:** You are in the wrong directory.

```
Run: cd D:\OneDrive\Desktop\lms\LMS-full-stack\lms-full-stack\server
Then: node server.js
```

### **Error: npm run dev fails with esbuild error**

**Cause:** node\_modules is corrupted.

```
cd client
rmdir /s /q node_modules
del package-lock.json
npm install
npm run dev
```

## 16. Manual Enrollment Script

Since the Stripe webhook may not auto-enroll users due to timestamp issues, use this script to manually complete all pending purchases. Run this from the server folder AFTER the server is stopped:

```
node --input-type=module --eval "
import mongoose from 'mongoose';
import dotenv from 'dotenv';
dotenv.config();
await mongoose.connect(process.env.MONGODB_URI);
const User = (await import('./models/User.js')).default;
const Course = (await import('./models/Course.js')).default;
const { Purchase } = await import('./models/Purchase.js');
const purchases = await Purchase.find({ status: 'pending' });
for (const purchase of purchases) {
  const user = await User.findById(purchase.userId);
  const course = await Course.findById(purchase.courseId);
  if (!user || !course) continue;
  const enrolled = user.enrolledCourses.some(id => id.toString() === course._id.toString());
  if (!enrolled) {
    await User.findByIdAndUpdate(user._id, { enrolledCourses: [...user.enrolledCourses, course._id] });
    await Course.findByIdAndUpdate(course._id, { enrolledStudents: [...course.enrolledStudents, user._id] });
  }
  await Purchase.findByIdAndUpdate(purchase._id, { status: 'completed' });
}
console.log('Done!'); process.exit();
"
```

Then restart the server and refresh the My Enrollments page.